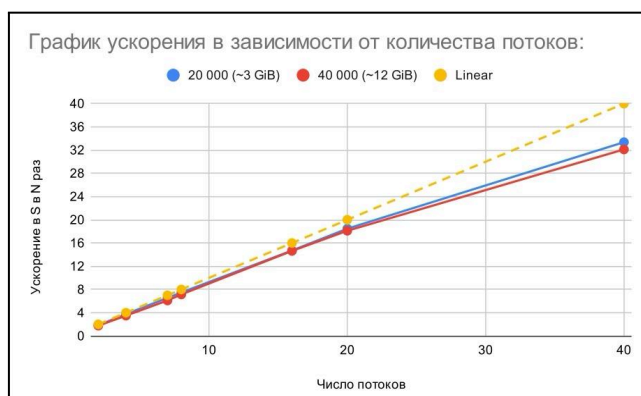


Выводы из результатов заданий:



1) Изучая график ускорения в зависимости от числа потоков, я смог выделить несколько ключевых наблюдений:

-> Ускорение для обеих матриц (20000x20000 и 40000x40000) следует схожему тренду, что и линейное выполнение, но для большей матрицы (красные точки) наблюдается чуть меньший рост ускорения при увеличении числа потоков, особенно в диапазоне 10–40 потоков. Это, возможно, связано с увеличенной нагрузкой на память во много раз, что может вызывать дополнительные накладные расходы.

До 10 потоков:

Ускорение растёт почти линейно, что говорит о хорошем использовании потоков. В этом диапазоне масштабируемость близка к идеальной, и никакого понижения производительности не наблюдается.

От 10 до 20 потоков:

Рост ускорения замедляется, начинает появляться эффект насыщения. Разница между синей и красной линией становится заметнее: большая матрица обрабатывается менее эффективно по сравнению с меньшей. Это может быть связано с ограничением памяти и межъядерного взаимодействия.

От 20 до 40 потоков:

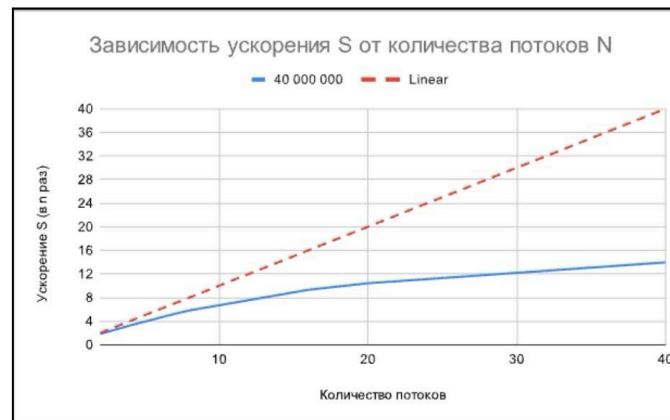
Ускорение продолжает расти, но всё дальше отстаёт от идеальной линейной зависимости. Особенно заметно расхождение с жёлтой пунктирной линией, что говорит о росте накладных расходов на координацию потоков и доступ к памяти. В точке 40 потоков ускорение достигает чуть более 30 раз, тогда как идеальное значение должно быть 40.

-> Выводы:

При увеличении количества потоков ускорение сначала растёт почти линейно, но затем начинает отставать из-за ограничений памяти, синхронизации потоков и накладных расходов.

Чем больше размер матрицы, тем сильнее эффект насыщения (на 40 000 x 40 000 рост медленнее, чем на 20 000 x 20 000).

После 20 потоков эффективность масштабирования снижается, а при 40 потоках видно существенное отклонение от идеального линейного ускорения.



2) На следующем графике представлено ускорение в зависимости от количества потоков. Судя по графику, можно сделать следующие выводы по локальным пересмотрам:

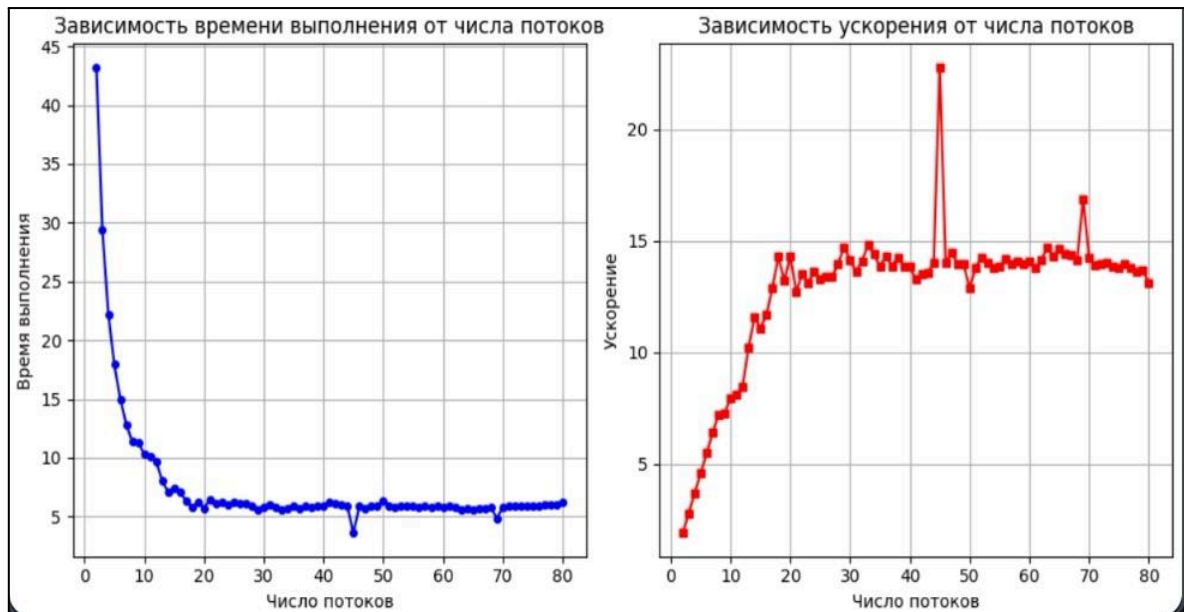
-> В диапазоне **до 10 потоков** ускорение растёт достаточно быстро, почти линейно, но уже немного отстаёт от идеального. Это говорит о хорошем использовании многопоточности, но с небольшими накладными расходами.

-> Затем после 10-20 потоков наблюдается Эффект насыщения: ускорение начинает замедляться. Это может быть связано с увеличением накладных расходов на синхронизацию потоков, взаимодействие с памятью и управлением вычислениями.

-> Наконец, ограничение ускорения при 40 потоках: При 40 потоках ускорение достигает чуть больше 10 раз вместо 40 (идеального значения). Это свидетельствует о существенном падении эффективности распараллеливания, возможно, из-за конкуренции потоков за доступ к памяти (общие переменные, кэш-память, NUMA-архитектура).

Насчёт `#pragma omp atomic`: хоть данная парадигма и помогает избегать “гонки потоков” за счёт атомарных операций, предотвращающих состояние гонки, когда несколько потоков одновременно изменяют `sum`, однако частые обновления `sum` вызывают задержки - атомарные операции всё же накладывают накладные расходы, особенно когда большое количество потоков одновременно обновляет `sum`, что, в свою очередь, ограничивает масштабируемость программы (её способность эффективно использовать увеличивающееся количество ресурсов для достижения лучшей производительности).

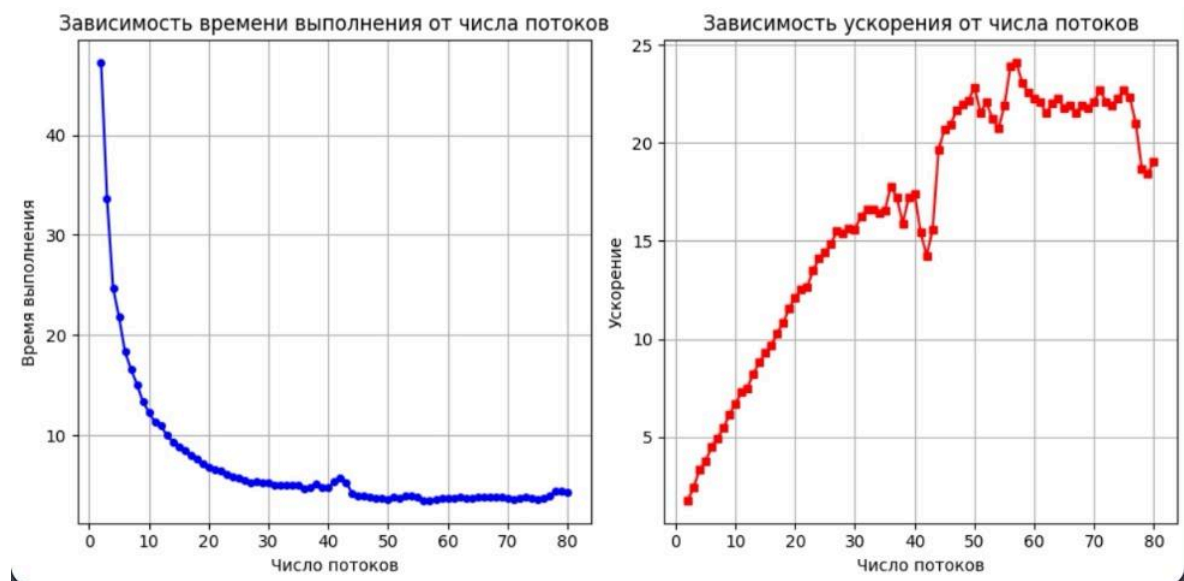
3) Третье задание было на анализ общего и полного распараллеливания алгоритма решения системы линейных уравнений, основанном на методе маленьких итераций. Начнём с графика полного распараллеливания:



Полное распараллеливание:

-> Общий анализ графика:

Время выполнения (синий график) резко снижается до ~40 потоков, после чего стабилизируется. Ускорение (красный график) имеет некоторые выбросы и колебания, но остается, в целом, на среднем (приемлемом) уровне.



Общее распараллеливание:

-> Общий анализ графика:

Время выполнения также быстро падает, но стабилизируется более плавно. Ускорение растёт равномернее, достигая более высоких значений (~20х).

-> Анализ эффективности:

- *Скорость работы:* Оба метода дают схожие результаты по снижению времени выполнения, но полное распараллеливание имеет более нестабильные точки (спады на 40 и 70 потоках).
- *Масштабируемость:* В общем распараллеливании ускорение увеличивается более равномерно и достигает более высоких значений без сильных выбросов.
- *Оптимальное число потоков:* В обоих случаях около 40 потоков даёт максимальное снижение времени выполнения.

-> Общий вывод:

Общее распараллеливание выглядит более эффективным, так как даёт более стабильное ускорение и лучше масштабируется. Полное распараллеливание приводит к нестабильностям, что может быть вызвано накладными расходами на создание и управление множественным числом потоков.